

텐서 구조체

3장에서 다루는 내용

- 파이토치 기본 자료구조인 텐서의 이해
- 텐서를 인덱스로 접근해서 연산하기
- 다차원 배열 넘파이와 연계해서 다루기
- 성능 개선을 위해 GPU로 연산 처리

2장에서 우리는 다양한 딥러닝 애플리케이션을 살펴보았다. 대부분의 이미지나 텍스트 같은 특정 형태의 입력 데이터를 처리하여 레이블이나 숫자 혹은 이미지나 텍스트 등의 여러 형태의 데이터로 만들었다. 이런 시각에서 살펴보면, 딥러닝은 데이터를 하나의 형태에서 다른 형태로 변환하는 시스템을 구축하는 과정을 포함한다고 볼 수 있다. 이런 변환은 원하는 매핑을 보여주기 위한 일련의 예제에서 공통점을 추출하는 방식으로 이뤄진다. 예를 들어 시스템은 학습 중에 일반적인 강아지의 전체적인 모양이나 골든 리트리버의 전형적인 색상 등을 기록해둔다. 그리고 이 두 가지 이미지 속성을 합쳐서 주어진 모양과 색상을 정확하게 이미지에 연결해 놓는다. 즉 골든 리트리버의 경우 블랙 레브라도 리트리버(혹은 황갈색 고양이)가 아닌 정확한 골든 리트리버의 모양과 색에 연결해 놓는다. 이런 방법으로 시스템은 유사한 입력의 다양한 사례를 받아들여 의미있는 출력을 만든다.

딥러닝 프로세스는 입력을 부동소수점 수로 변환하는 것부터 시작한다. 4장에서는 그림 3.1의 첫 단계에 볼 수 있는 이미지 픽셀을 숫자로 변환하는 작업을 수행할 것이다. 이런 과정을 살펴 보기에 앞서, 3장에서는 텐서를 사용해 파이토치가 부동소수점 수를 어떻게 다루는지를 먼저 알아본다.

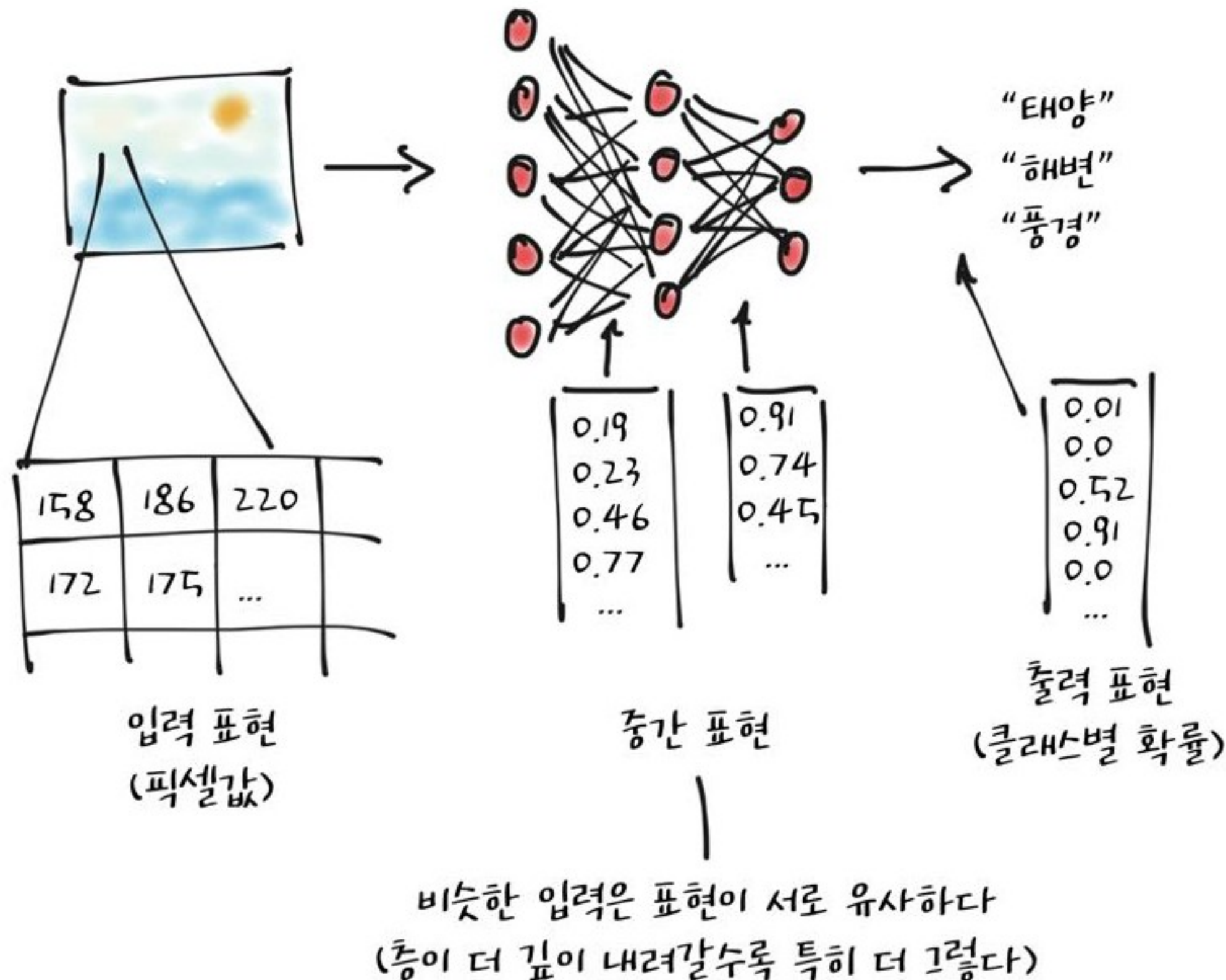


그림 3.1 입력을 출력으로 바꾸는 법을 배우는 심층 신경망 (주의: 뉴런 개수와 출력은 확장 가능하지 않다)

3.1 부동소수점 수의 세계

신경망 안에서의 정보 처리는 결국 부동소수점 수로 나타내므로 우리가 원하는 실세계의 데이터를 신경망이 연산 가능하도록 인코딩해야 하고, 처리 후 나온 결과값은 해석 가능하거나 사용할 수 있도록 디코딩해야 한다.

심층 신경망^{deep neural netowrk}은 보통 여러 단계를 거쳐 데이터 변환을 학습한다. 그래서 각 단계 사이의 일부 변환된 데이터들은 중간 단계를 표현하는 연속적인 흐름으로 생각할 수 있다. 이미지 인식의 경우 앞 단계에서는 모서리를 검출하거나 털 같은 질감을 잡아내고 깊이 들어갈수록 귀나 코 혹은 눈 같은 복잡한 구조를 파악한다.

일반적으로 이러한 중간 단계는 입력값의 특징을 잡아내는 부동소수점 수의 모음인 동시에 신경망에서 입력이 최종적으로 출력으로 표현되는 방법을 기술하기 위한 수단으로 데이터 구조를 잡아낸다. 중간 표현값은 부동소수점 수로 이루어져 있고 입력을 특징짓는 동시에 데이터 구조를 잡아내어 신경망 출력과 어떻게 매핑되는지를 설명하는 데 중요한 역할을 한다. 특징을 잡아내는 것은 작업에 따라 고유하며 적절한 예제를 통해 학습된다. 생성된 부동소수점 수의 모음과 이 값의 조작법이 AI의 핵심이며 앞으로 이 책 전반에 걸쳐 다양한 사례를 보게 될 것이다.

한 가지 염두에 두면 좋을 중요한 내용은 이러한 중간 표현값(그림 3.1의 두 번째 단계에서 보이는 부분)은 입력과 이전 층의 뉴런이 가진 가중치를 조합한 결과라는 점이다. 중간 단계의 개별 표현은 자신만의 방식으로 앞 단계에서 넘어온 입력을 변환한다.

우리의 데이터를 부동소수점 입력으로 바꾸기 전에 먼저, 파이토치가 입력과 중간 표현 그리고 출력으로서 어떻게 데이터를 다루고 저장하는지를 잘 이해할 필요가 있다. 이게 바로 이 장에서 제대로 다룰 내용이다.

데이터 처리와 저장을 위해 파이토치에서는 **텐서**^{tensor}라는 기본 자료구조를 제공한다. 사전 훈련된 신경망으로 추론 작업을 수행하는 과정을 다룬 2장에서 이미 우리는 텐서를 살펴봤다. 수학이나 물리학 혹은 공학을 전공했다면 공간 개념과 참조^{reference} 시스템, 그리고 이 둘 사이의 변환을 수행할 때 들어본 개념일 텐데, 여기서 말하는 텐서는 그런 개념과는 다르다. 딥러닝에서의 텐서는 그림 3.2에서 보는 것과 같은 임의의 차원을 가진 벡터나 행렬의 일반화된 개념으로 생각하면 된다. 텐서를 **다차원 배열**^{multidimensional array}이라고도 부른다. 텐서의 차원 수는 텐서 안의 스칼라 값을 참조하기 위해 사용하는 인덱스 수와 동일하다.

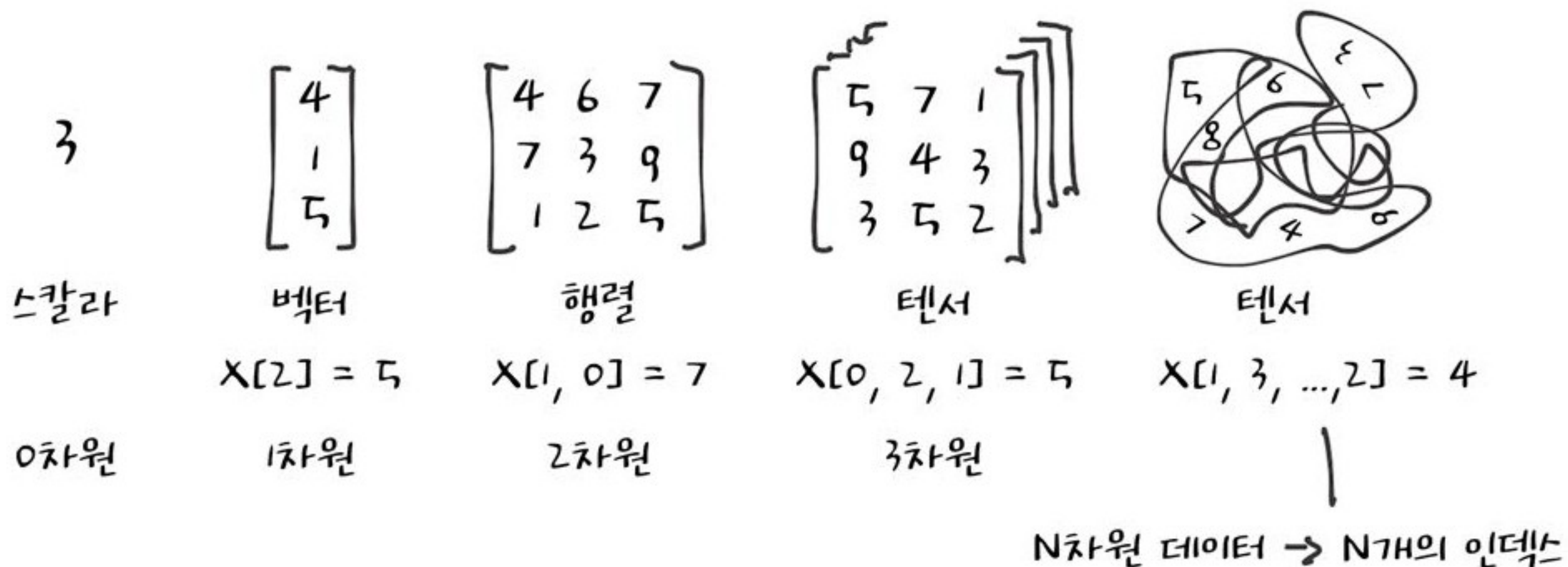


그림 3.2 텐서는 파이토치에서 데이터를 표현하기 위한 빌딩 블록이다.

파이토치가 다차원 배열을 다루는 유일한 라이브러리는 아니다. 데이터과학 분야의 표준이나 마찬가지로 넘파이NumPy가 현재까지는 제일 유명한 다차원 배열 라이브러리다. 파이토치는 넘파이와 깔끔하게 호환되도록 만들어져 있기 때문에 사이파이SciPy(www.scipy.org)나 사이킷런Scikit-learn(scikit-learn.org), 판다스Pandas(pandas.pydata.org) 같은 과학 라이브러리와 자연스럽게 통합된다.

넘파이 배열과 비교하면 파이토치 텐서가 조금 낫다. 예를 들어 GPU 성능 최적화나 여러 장비나 컴퓨터를 사용한 분산 실행 혹은 연산 그래프 추적 등을 꼽을 수 있다. 이런 특징은 오늘날의 딥러닝 라이브러리를 구현할 때 중요한 기능으로 꼽힌다.

그럼 이제 먼저 파이토치 텐서의 기본적인 소개부터 시작하자. 그다음에는 책 전반에 걸쳐 수행할 작업을 위한 기초 내용을 다룰 것이다. 우선, 파이토치 텐서 라이브러리를 사용해 텐서를 조작하는 법을 배워보자. 데이터가 메모리에 어떻게 보관되는지, 엄청나게 큰 텐서에 대해 특정 연산을 어떻게 상수 시간 안에 수행하는지, 그리고 앞에서 언급한 넘파이와의 연동 및 GPU 가속 기능도 함께 알아보자. 또한, 앞으로 우리의 주력 도구로 텐서를 사용하게 되므로 텐서 API가 할 수 있는 것을 파악하는 것도 필요하다. 다음 4장에서는 이 장에서 배운 내용을 잘 활용해 다양한 데이터를 신경망이 배울 수 있도록 표현하는 법을 배워볼 것이다.

3.2 텐서: 다차원 배열

텐서는 파이토치의 기본 자료구조^{data structure}임을 앞서 살펴보았다. 텐서는 일종의 배열이다. 즉 한 개나 여러 개의 인덱스를 사용해 개별적으로 값에 접근할 수 있는 형태의 숫자 모음을 저장하는 자료구조다.

3.2.1 파이썬 리스트에서 파이토치 텐서로

텐서 인덱싱과 비교하기 위해 우선 list 인덱싱을 실제로 해보자. 숫자 세 개를 가진 리스트를 파이썬으로 만들자([code/p1ch3/1_tensors.ipynb](#)).

```
# In[1]:  
a = [1.0, 2.0, 1.0]
```

리스트의 첫 번째 요소를 0부터 시작하는 인덱스로 접근할 수 있다.

```
# In[2]:
```

```
a[0]
```

```
# Out[2]:
```

```
1.0
```

```
# In[3]:
```

```
a[2] = 3.0
```

```
a
```

```
# Out[3]:
```

```
[1.0, 2.0, 3.0]
```


파이썬 프로그램이 2차원에서 선의 좌표 같은 숫자 벡터를 다루기 위해 파이썬 리스트를 사용하는 일은 흔하다. 다음 장에서 보게 되겠지만 텐서 자료구조를 사용해 이미지와 시계열 데이터 혹은 문장들을 나타내는 것이 더 효율적이다. 이번 장에서 간간히 살펴볼 텐데, 텐서에 대해 연산을 정의해 두면 동일한 작업에 대해 파이썬 같은 고차원 언어(지만 빠르지는 않은)보다 훨씬 효율적이고 알아보기 쉽게 데이터를 자르고 조작할 수 있다.

3.2.2 첫 텐서 만들어보기

우선 파이토치로 텐서를 만든 다음, 찬찬히 한번 살펴보자. 아직은 그렇게 대단한 텐서를 만드는 것은 아니지만, 일단 세 개 열에 1이 들어 있는 간단한 텐서를 만들어보자.

```
# In[4]:
import torch  ← torch 모듈 임포트하기
a = torch.ones(3)  ← 크기가 3인 1차원 텐서를 만들고 값을 1로 채우기
a
```

```
# Out[4]:
tensor([1., 1., 1.])
```

```
# In[5]:
a[1]
```

```
# Out[5]:
tensor(1.)
```

```
# In[6]:
float(a[1])
```

```
# Out[6]:
1.0
```

torch 모듈을 임포트하고 나면 1.0이라는 값으로 채워진, 크기가 3인 (1차원) 텐서를 만드는 함수를 호출할 수 있다. 0부터 시작하는 인덱스를 사용하여 값을 읽거나 새로운 값을 대입할 수 있다. 겉으로는 숫자 객체의 리스트처럼 보이지만 내부 동작은 완전히 다르다.

```
# In[7]:
a[2] = 2.0
a
```

```
# Out[7]:
tensor([1., 1., 2.])
```

3.2.3 텐서의 핵심

숫자값으로 만든 파이썬 리스트나 튜플 객체는 그림 3.3의 왼쪽 그림처럼 메모리에 따로따로 할당된다. 반면 파이토치 텐서나 넘파이 배열은 파이썬 객체가 아닌 **언박싱**unboxing된 C 언어의 숫자 타입을 포함한 연속적인 메모리가 할당되고 이에 대한 뷰를 제공한다. 여기서 각 요소는 32비트(4바이트) float 타입이며 그림 3.3의 오른쪽 그림에서 볼 수 있다. 100만 개의 float 타입 숫자를 1차원 텐서에 보관한다면 400만 바이트의 연속적인 공간과 (차원이나 숫자 타입을 기록하는 용도의) 메타데이터 공간을 조금 더 차지한다.

위치 객체를 나타내는 좌표 리스트가 있다고 생각해보자. 즉 각 좌표 (4, 1), (5, 3), (2, 1)이 2차원 삼각형의 꼭짓점에 각각 대응된다고 생각해보자. 딥러닝 예제로 적절하지는 않지만 이해하기는 쉽다. 파이썬 리스트로 숫자 좌표를 유지하는 대신, 짝수 인덱스는 X 에, 홀수 인덱스는 Y 에 넣는 식으로 1차원 텐서를 사용할 수 있다.

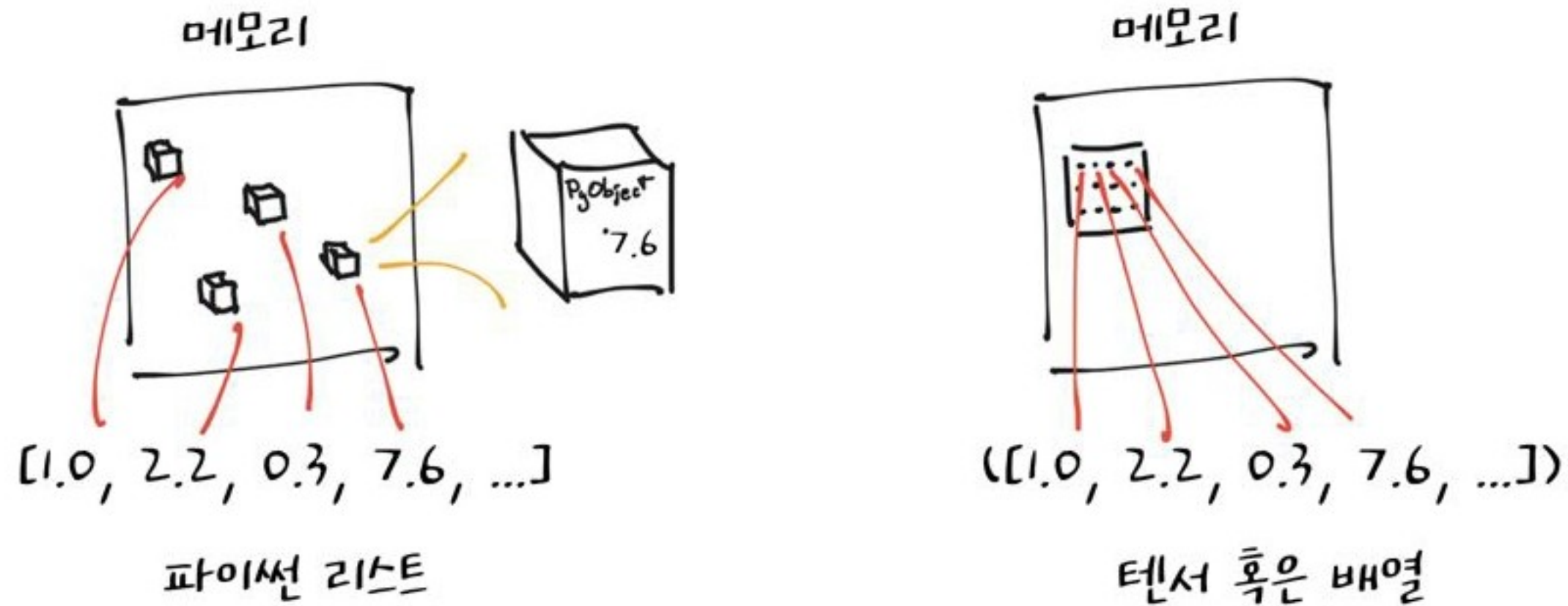


그림 3-3 (박싱된) 파이썬 객체 숫자값과 (언박싱 배열인) 텐서 숫자값

```
# In[8]:
points = torch.zeros(6)  ←—.zeros를 사용하면 쉽게 원하는 크기의 배열을 얻음
points[0] = 4.0  ←—.그리고 나서 원하는 값으로 모든 0을 덮어씀
points[1] = 1.0
points[2] = 5.0
points[3] = 3.0
points[4] = 2.0
points[5] = 1.0
```

생성자에 파이썬 리스트를 넘겨도 된다.

```
# In[9]:
points = torch.tensor([4.0, 1.0, 5.0, 3.0, 2.0, 1.0])
points
```

```
# Out[9]:
tensor([4., 1., 5., 3., 2., 1.])
```

첫 번째 점의 좌표를 읽으려면 다음과 같이 한다.

```
# In[10]:  
float(points[0]), float(points[1])
```

```
# Out[10]:  
(4.0, 1.0)
```

이 정도도 좋지만 좌표 대신 2차원 좌표를 바로 인덱싱하는 게 더 실용적이니 2차원 텐서를 사용하자.

```
# In[11]:  
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])  
points
```

```
# Out[11]:  
tensor([[4., 1.],  
        [5., 3.],  
        [2., 1.]])
```

이번에는 리스트의 리스트를 생성자로 넘겨주고 텐서의 차원을 살펴보자.

```
# In[12]:  
points.shape
```

```
# Out[12]:  
torch.Size([3, 2])
```

이렇게 하면 각 차원과 함께 텐서의 크기도 알 수 있다. 텐서 초기화를 위해 차원별 크기 정보를 튜플로 만들어 zeros나 ones로 넘겨줄 수도 있다.

```
# In[13]:  
points = torch.zeros(3, 2)  
points
```

```
# Out[13]:  
tensor([[0., 0.],  
        [0., 0.],  
        [0., 0.]])
```

이제 두 개의 인덱스로 텐서 내 요소를 개별적으로 접근할 수 있다.


```
# In[14]:  
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])  
points
```

```
# Out[14]:  
tensor([[4., 1.],  
        [5., 3.],  
        [2., 1.]])
```

```
# In[15]:  
points[0, 1]
```

```
# Out[15]:  
tensor(1.)
```

이렇게 데이터셋의 0번째 포인트의 Y 좌표를 반환한다. 혹은 앞에서 했던 것처럼 첫 번째 포인트의 2차원 좌표를 텐서에서 얻어낼 수도 있다.

```
# In[16]:
```

```
points[0]
```

```
# Out[16]:
```

```
tensor([4., 1.])
```

여기서는 동일한 데이터를 다른 형태의 뷰로 표현하는 또 다른 텐서를 보여준다. 새 텐서는 길이가 2인 1차원 텐서인데 `points` 텐서 안에 첫째 행의 값들을 참조한다. 이렇게 할 때 새로운 메모리 영역이 할당되고, 값이 복사된 후 새 메모리에 대해 새 텐서 객체가 래핑되어 반환되는 것일까? 그렇지 않다. 포인트가 수백만 개일 경우 새 메모리를 할당하는 방식은 매우 비효율적이기 때문이다. 텐서가 어떻게 저장되는 지는 3.7절에서 텐서 뷰와 함께 설명한다.

3.3 텐서 인덱싱

만일 모든 포인트에서 첫 번째 값만 구해야 한다면? 다음처럼 표준 파이썬과 마찬가지로 범위 인덱싱을 사용한다. 이제 복습을 한번 해보자.

```
# In[53]:
some_list = list(range(6))
some_list[:]  ← 리스트의 모든 요소
some_list[1:4] ← 1번 인덱스에 있는 요소부터 4번 인덱스 전까지
some_list[1:]  ← 1번 인덱스에 있는 요소부터 끝까지
some_list[:4]  ← 리스트의 첫 번째 요소부터 4번 인덱스 전까지
some_list[:-1] ← 리스트의 첫 번째 요소부터 마지막 요소 바로 앞까지
some_list[1:4:2] ← 1번 인덱스 요소부터 4 번 인덱스 앞까지, 두 단계씩 건너뛰면서
```

파이토치 텐서도 넘파이를 비롯한 파이썬 과학 라이브러리와 동일한 표기법을 사용할 수 있고 텐서의 차원도 마찬가지다.

```
# In[54]:
points[1:]  ← 첫 번째 이후 모든 행에 대해. 암묵적으로 모든 열이 포함됨
points[1:, :] ← 첫 번째 이후 모든 행에 대해. 명시적으로 모든 열이 포함됨
points[1:, 0] ← 첫 번째 이후 모든 행에 대해 첫 번째 열만 포함됨
points[None] ← 길이가 1인 차원을 추가함. unsqueeze와 동일함
```

범위 인덱싱과 더불어 파이토치는 **고급 인덱싱** advanced indexing이라 부르는 더 강력한 방식도 지원하는데 이에 대해서는 4장에서 살펴볼 것이다.

3.4 이름이 있는 텐서

우리가 다루는 텐서는 차원이나 축이 있으며, 각 차원은 픽셀 위치나 컬러 채널에 해당한다. 때문에 텐서를 접근하려면 차원의 순서를 기억해서 인덱싱해야 한다. 데이터가 여러 텐서 형태를 거치며 다양하게 변환되면, 어느 차원에 어떤 데이터가 들어있는지 헷갈려 실수하기 쉽다.

예를 들어 2.1.4절에서 봤던 `img_t` 3차원 텐서를 생각해보자(편의상 데이터는 아무 숫자나 넣었다). 이미지 데이터를 흑백으로 변환해야 한다고 가정해보자. 여러 색상별 가중치를 보고 하나의 밝기 값을 뽑아내는 과정이 된다.¹

```
# In[2]:  
img_t = torch.randn(3, 5, 5) # 각각이 [채널 크기, 행 크기, 열 크기]가 됨  
weights = torch.tensor([0.2126, 0.7152, 0.0722])
```

이런 코드는 일반화하면 좋다. 예를 들어 높이와 너비를 가진 2차원 텐서로 이뤄진 흑백 이미지로부터 RGB값을 담은 세 번째 채널 차원을 더하는 코드로 만드는 것이다. 아니면 개별 이미지 여러 개를 묶어 배치로 만드는 코드를 생각할 수 있다. 2.1.4절의 `batch_t`에서 추가적인 배치 차원에 대해 소개했다. 지금은 배치 크기 2로 가정하자.


```
# In[3]:
```

```
batch_t = torch.randn(2, 3, 5, 5) # [채널 크기, 행 크기, 열 크기]
```

경우에 따라 RGB 채널은 0번 혹은 1번 차원에 있다. 두 경우 모두 뒤에서부터 세 번째 차원이므로 RGB 채널은 -3번 차원에 있는 것으로 일반화할 수 있다. 이러면 다음 코드로 배치 차원 유무에 상관없이 평균을 구할 수 있다.

```
# In[4]:
```

```
img_gray_naive = img_t.mean(-3)
```

```
batch_gray_naive = batch_t.mean(-3)
```

```
img_gray_naive.shape, batch_gray_naive.shape
```

```
# Out[4]:
```

```
(torch.Size([5, 5]), torch.Size([2, 5, 5]))
```

지금은 가중치가 있다. 파이토치는 동일한 차원 정보의 텐서끼리 연산할 수 있고, 각 차원의 길이가 1인 텐서도 가능하다. 혹은 길이가 1인 차원을 알아서 늘려주기도 하는데, 이런 방식을 **브로드캐스팅**^{broadcasting}이라 한다. (2, 3, 5, 5) 차원을 가진 batch_t를 (3, 1, 1) 차원의 unsqueezed_weights로 곱하면 (2, 3, 5, 5) 차원이 되고, 채널 정보를 가진, 뒤에서 세 번째 차원에 있는 값에 대한 합을 구할 수 있다.²

```
# In[5]:
```

```
unsqueezed_weights = weights.unsqueeze(-1).unsqueeze_(-1)
img_weights = (img_t * unsqueezed_weights)
batch_weights = (batch_t * unsqueezed_weights)
img_gray_weighted = img_weights.sum(-3)
batch_gray_weighted = batch_weights.sum(-3)
batch_weights.shape, batch_t.shape, unsqueezed_weights.shape
```

```
# Out[5]:
```

```
(torch.Size([2, 3, 5, 5]), torch.Size([2, 3, 5, 5]), torch.Size([3, 1, 1]))
```

금세 코드가 복잡해졌다. 이런 경우에 효율성 차원에서 파이토치는 넘파이에서 차용한 einsum 함수를 제공한다. 차원별로 이름을 부여하는 작은 언어³라고 생각하면 되는데 이와 같이 곱셈 후 합을 구하는 경우에 사용할 수 있다. 파이썬에서는 점 세 개(...)로 변수명 없이 합을 구하는 브로드캐스팅을 사용하는데 다음 코드에서는 einsum을 사용하지 않을 테니 einsum에 대해서는 너무 걱정하지 말자.

```
# In[6]:
img_gray_weighted_fancy = torch.einsum('...chw,c->...hw', img_t, weights)
batch_gray_weighted_fancy = torch.einsum('...chw,c->...hw', batch_t, weights)
batch_gray_weighted_fancy.shape

# Out[6]:
torch.Size([2, 5, 5])
```

보다시피 많은 기호가 복잡하게 사용되고 있다. 혹여라도 텐서 생성 코드로부터 멀리 떨어진 위치에서 텐서를 사용하는 코드를 작성하면 헛갈려서 오류를 내기 쉽다. 그래서 전문가들은 각 차원에 이름을 부여하는 방식을 제안해왔다.⁴

파이토치 1.3 버전에서는 실험적인 기능으로 이름 지정이 가능한 텐서가 등장했다(https://pytorch.org/tutorials/intermediate/named_tensor_tutorial.html와 https://pytorch.org/docs/stable/named_tensor.html를 보라). `tensor`나 `rand` 같은 텐서 팩토리 함수에 이름으로 사용할 문자열 리스트를 `names` 인자로 전달할 수 있다.

```
# In[7]:
weights_named = torch.tensor([0.2126, 0.7152, 0.0722], names=['channels'])
weights_named

# Out[7]:
tensor([0.2126, 0.7152, 0.0722], names=('channels',))
```

텐서를 먼저 만들고 나중에 이름을 지정하려면(만들어놓은 텐서를 새로 바꾸기 싫다면), `redefine_names` 함수를 사용한다. 텐서 접근 시 인덱싱하듯이 생략 부호 `...`를 사용하면 다른 차원은 건드리지 않는다. 비슷한 함수인 `rename`을 사용하면 이미 있는 이름을 바꿀 수 있고, 지우려면 이름으로 `None`을 넘긴다.


```
# In[8]:
img_named = img_t.refine_names(..., 'channels', 'rows', 'columns')
batch_named = batch_t.refine_names(..., 'channels', 'rows', 'columns')
print("img named:", img_named.shape, img_named.names)
print("batch named:", batch_named.shape, batch_named.names)

# Out[8]:
img named: torch.Size([3, 5, 5]) ('channels', 'rows', 'columns')
batch named: torch.Size([2, 3, 5, 5]) (None, 'channels', 'rows', 'columns')
```

텐서끼리의 연산은 먼저 각 차원의 크기가 같은지 혹은 한쪽이 1이고 다른 쪽으로 브로드캐스팅될 수 있는지도 확인해야 한다. 이때 이름이 지정되어 있다면 파이토치가 우리를 대신해 알아서 체크해줄 것이다. 파이토치가 차원을 자동으로 정렬해주지는 않기 때문에 우리는 명시적으로 이런 작업을 수행할 필요가 있다. `align_as` 함수는 빠진 차원을 채우고, 존재하는 차원을 올바른 순서로 바꿔준다.

```
# In[9]:
weights_aligned = weights_named.align_as(img_named)
weights_aligned.shape, weights_aligned.names
```

```
# Out[9]:
(torch.Size([3, 1, 1]), ('channels', 'rows', 'columns'))
```

sum처럼 차원 인수를 허용하는 함수들은 이름이 붙은 차원도 받아들인다.

```
# In[10]:
gray_named = (img_named * weights_aligned).sum('channels')
gray_named.shape, gray_named.names
```

```
# Out[10]:
(torch.Size([5, 5]), ('rows', 'columns'))
```

이름이 다른 차원을 결합하려면 오류가 발생한다.

```
gray_named = (img_named[..., :3] * weights_named).sum('channels')
```

```
RuntimeError: Error when
attempting to broadcast dims ['channels', 'rows',
'columns'] and dims ['channels']: dim 'columns' and dim 'channels'
are at the same position from the right but do not match.
```

이름 있는 텐서를 사용하는 연산을 함수 밖에서도 사용하려면, 다음처럼 차원 이름에 `None`을 넣어 이름 없는 텐서를 만든다.

```
# In[12]:  
gray_plain = gray_named.rename(None)  
gray_plain.shape, gray_plain.names  
  
# Out[12]:  
(torch.Size([5, 5]), (None, None))
```

현재 이 기능은 아직 실험 단계이며, 인덱싱과 정렬 작업을 수행하느라 헛수고하지 않기 위해 이 책에서는 텐서에 이름을 지정하지 않고 사용한다. 이름 있는 텐서는 파이토치 포럼에서 자주 언급되는 정렬 오류 같은 잠재적인 골칫거리를 제거하는 효과가 있으므로 앞으로 사람들이 얼마나 활용할지 기대가 된다.

3.5 텐서 요소 타입

지금까지 텐서의 기본 동작은 살펴봤지만 아직 Tensor에 어떤 타입의 값을 저장할 수 있는지는 설명하지 않았다. 3.2절의 힌트처럼 표준 파이썬 숫자 타입은 여러 이유로 최적은 아니다.

- 파이썬에서 숫자는 객체다. 통상 부동소수점 수는 컴퓨터에서 32비트 공간을 사용한다. 하지만 파이썬은 참조 카운터까지 만들어 부동소수점 수를 완전한 파이썬 객체로 변환한다. 박싱^{boxing}이라 부르는 이 연산은 수를 소량만 저장하는 경우라면 큰 문제가 없겠지만, 수 백만 개가 넘어가면 상당히 비효율적이다.
- 파이썬에서 리스트는 연속된 객체의 컬렉션이다. 파이썬은 두 벡터의 내적을 효율적으로 수행하는 연산이 없다. 벡터 합도 마찬가지다. 파이썬 리스트에 들어있는 데이터를 메모리에 최적화하여 배치할 방법은 딱히 없고, 리스트는 숫자 뿐만 아니라 임의의 파이썬 객체에 대한 임의 접근이 가능한 포인터의 모음이다. 게다가 파이썬 리스트는 단일 차원이며, 비록 리스트의 리스트를 만들 수도 있지만 이런 방식은 매우 비효율적이다.
- 파이썬 인터프리터는 최적화를 거치는 컴파일된 코드보다 느리다. 다량의 숫자 데이터 모음에 대한 수학적 연산을 수행하는 일은 C 같은 저수준 컴파일을 통해 최적화한 바이너리 코드가 훨씬 빠르다.

이런 이유 때문에 데이터과학 라이브러리는 넘파이에 의존하거나 파이토치 텐서같이 전용 데이터 구조를 만든 후 숫자 데이터 연산은 저수준 언어로 효율을 높이도록 구현하고 동시에 고차원 API로 이런 구현을 래핑하여 편리성을 더한다. 성능 최적화를 위해 텐서 내의 모든 객체는 같은 타입의 숫자여야 하고 파이토치는 실행 중에 이런 숫자 타입을 계속 추적하고 있어야 한다.

3.5.1 dtype으로 숫자 타입 지정하기

tensor나 zeros, ones 같은 텐서 생성자 실행 시 넘겨주는 dtype 인자로 텐서 내부에 들어갈 데이터 타입(data type을 줄여 dtype라 함)을 지정할 수 있다. 이를 통해 텐서가 가질 값이 정수 혹은 부동소수점 수 같은 타입을 지정하고 각 값이 차지하는 바이트 수도 명세한다.⁵ dtype 인자는 표준 넘파이 인자와 거의 동일하다. dtype 인자 타입을 열거하면 다음과 같다.

- torch.float32 혹은 torch.float: 32비트 단정밀도 부동소수점
- torch.float64 혹은 torch.double: 64비트 배정밀도 부동소수점
- torch.float16 혹은 torch.half: 16비트 반정밀도 부동소수점
- torch.int8: 부호 있는 8비트 정수
- torch.uint8: 부호 없는 8비트 정수
- torch.int16 혹은 torch.short: 부호 있는 16비트 정수
- torch.int32 혹은 torch.int: 부호 있는 32비트 정수
- torch.int64 혹은 torch.long: 부호 있는 64비트 정수
- torch.bool: 불리언

텐서의 기본 데이터 타입은 32비트 부동소수점이다.

3.5.2 모든 경우에 사용하는 dtype

이 책 전반에서 살펴보겠지만, 신경망 연산은 대부분 32비트 부동소수점 연산이다. 배정밀도 64비트를 사용해봐야 모델의 정확도는 거의 개선되지 않고 더 많은 메모리와 시간만 낭비한다. 16비트 반정밀도 부동소수점은 표준 CPU 구현은 찾기 힘들지만 최신 GPU에는 내부에 구현되어 있다. 필요하다면 약간의 정확도를 희생해서 정밀도를 반으로 떨어뜨려 신경망이 차지하는 공간을 줄이는 방식도 가능하다.

텐서는 다른 텐서에 대한 인덱스로 사용할 수 있다. 이때 파이토치는 인덱싱용 텐서를 64비트 정수 데이터 타입으로 간주한다. 텐서를 만들 때 `torch.tensor([2, 2])`처럼 인자로 정수값을 주면 64비트 정수 텐서를 기본으로 만든다. 앞으로 대부분의 경우 `float32`나 `int64` 값을 다룬다.

마지막으로 `points > 1.0` 같은 술어(predicate)는 텐서 내 각각이 이 조건을 만족하는지 알려주는 `bool` 텐서를 만들어낸다. 생성된 텐서 내 값은 숫자 타입이다.

3.5.3 텐서의 dtype 속성 관리

숫자 타입이 올바르게 지정된 텐서를 하나 할당할 때에는 생성자에 dtype 인자를 정확하게 전달해야 한다. 예를 들면 다음과 같다.

```
# In[47]:  
double_points = torch.ones(10, 2, dtype=torch.double)  
short_points = torch.tensor([[1, 2], [3, 4]], dtype=torch.short)
```

어떤 텐서가 가진 dtype을 알고 싶다면 다음과 같이 대응하는 속성을 읽어보면 된다.

```
# In[48]:  
short_points.dtype
```

```
# Out[48]:  
torch.int16
```


텐서 생성 함수가 반환하는 텐서의 타입을 대응하는 캐스팅 메소드를 사용해 올바른 타입으로 변환하는 것도 가능하다.

```
# In[49]:  
double_points = torch.zeros(10, 2).double()  
short_points = torch.ones(10, 2).short()
```

혹은 to 메소드를 사용하면 더 편하다.

```
# In[50]:  
double_points = torch.zeros(10, 2).to(torch.double)  
short_points = torch.ones(10, 2).to(dtype=torch.short)
```

to 메소드는 변환이 필요한 경우에만 진행한다. float 같은 dtype 이름을 사용한 캐스팅은 to를 사용한 것보다 짧게 쓸 수 있지만 to 메소드는 타입 외에도 추가적인 인자를 지정할 수 있다.

자세한 내용은 3.9절에서 다룬다.

여러 타입을 가진 입력들이 연산을 거치며 서로 섞일 때 자동으로 제일 큰 타입으로 만들어진다.
따라서 만약 32비트 연산을 원한다면 입력 타입 중 가장 큰 것이 32비트인지 확인하기 바란다.

```
# In[51]:
points_64 = torch.rand(5, dtype=torch.double)  ← rand는 텐서 요소를 0과 1 사이 임의의 수로 초기화함
points_short = points_64.to(torch.short)
points_64 * points_short  # 파이토치 1.3부터 쓸 수 있음

# Out[51]:
tensor([0., 0., 0., 0., 0.], dtype=torch.float64)
```

3.6 텐서 API

이제 우리는 파이토치 텐서가 무엇이며 내부 동작이 어떤지도 알게 됐다. 마무리하기 전에, 파이토치가 제공하는 텐서 연산을 훑어보자. 이 책에 모든 연산을 나열하는 일은 별로 쓸모가 없을 테다. 그보다는 API 전체에 대한 감을 잡아 나중에 <http://pytorch.org/docs> 온라인문서의 어디를 찾으면 자세한 내용을 볼 수 있는지를 파악해두자.

텐서끼리의 연산 대부분은 torch 모듈에 있고 대부분이 텐서 객체에 대해 메소드처럼 호출할 수 있다. 예를 들어 앞에서 본 transpose 함수도 torch 모듈로 호출한다.

```
# In[71]:  
a = torch.ones(3, 2)  
a_t = torch.transpose(a, 0, 1)  
  
a.shape, a_t.shape  
  
# Out[71]:  
(torch.Size([3, 2]), torch.Size([2, 3]))
```

혹은 텐서 메소드로 다음과 같이 할 수 있다.

```
# In[72]:  
a = torch.ones(3, 2)  
a_t = a.transpose(0, 1)  
  
a.shape, a_t.shape  
  
# Out[72]:  
(torch.Size([3, 2]), torch.Size([2, 3]))
```

두 코드는 동일하므로 아무 쪽이나 사용해도 된다.

앞서 언급한 파이토치 온라인 문서(<http://pytorch.org/docs>)에 텐서 연산이 완벽하게 잘 정리되어 있다. 해당 문서는 다음과 같이 구성되어 있다.

- Creation ops: ones나 from_numpy같이 텐서를 만드는 함수
- Indexing, slicing, joining, mutating ops: 셰이프^{shape}, 스트라이드^{stride}, transpose처럼 텐서 내부를 바꾸는 함수
- Math ops: 연산을 통해 텐서 내부를 조작하는 함수
 - Pointwise ops: abs나 cos처럼 텐서 요소 하나 하나에 대한 함수 실행 결과로 새 텐서를 만드는 함수
 - Reduction ops: mean, std, norm처럼 여러 텐서를 순회하며 집계하는 함수
 - Comparison ops: equal이나 max처럼 텐서 요소에 대해 참거짓을 숫자로 평가해서 반환하는 함수
 - Spectral ops: stft나 hamming_window처럼 주파수 영역에 대해 변환이나 연산을 수행하는 함수
 - Other operations: cross처럼 벡터에 대한 연산을 수행하거나, trace처럼 행렬에 대한 특수 연산을 실행하는 함수
 - BLAS and LAPACK operations: 기본 선형 대수 서브프로그램^{BLAS, Basic Linear Algebra Subprogram} 정의를 따르며 스칼라, 벡터-벡터, 행렬-벡터, 행렬-행렬에 대해 연산을 수행할 수 있는 함수
- Random sampling: randn이나 normal처럼 확률 분포에 기반해서 난수를 만드는 함수
- Serialization: load와 save처럼 텐서를 저장하거나 불러오는 함수
- Parallelism: set_num_threads처럼 병렬 CPU 처리 시 스레드 수를 제어하는 함수

3.7 텐서를 저장소 관점에서 머릿속에 그려보기

텐서의 내부 구조를 좀더 자세하게 살펴보자. 텐서 내부 값은 실제로는 `torch.Storage` 인스턴스로 관리하며 연속적인 메모리 조각으로 할당된 상태다. 저장 공간은 숫자 데이터를 가진 1차원 배열이다. 부동소수점 수를 나타내기 위한 32비트 공간의 `float`나 정수를 표현하기 위한 64비트 공간의 `int64` 타입으로 된 숫자들이 연속해서 들어 있는 메모리 블록으로 보면 된다. 파이토치의 `Tensor` 객체는 이러한 저장 공간을 나타내는 `Storage` 객체에 대한 뷰 역할을 담당하고, 오프셋을 사용해서 공간의 임의 위치에 접근하거나 특정 차원의 크기를 단위로 해서 접근할 수 있다.⁶

서로 다른 방식으로 구성된 텐서가 동일한 메모리 공간을 가리키고 있을 수도 있다. 그림 3.4의 예를 살펴보자. 3.2절에서 `points[0]`을 요청했을 때, `points` 텐서의 저장 공간을 가리키는 다른 텐서를 반환하는데 하나는 1차원이고 다른 하나는 2차원이다. 메모리는 한 번만 할당되었지만 동일한 데이터에 대해 다른 텐서 뷰를 만드는 작업은 `Storage` 객체가 관리하는 데이터 크기에 상관없이 빠르게 수행된다.

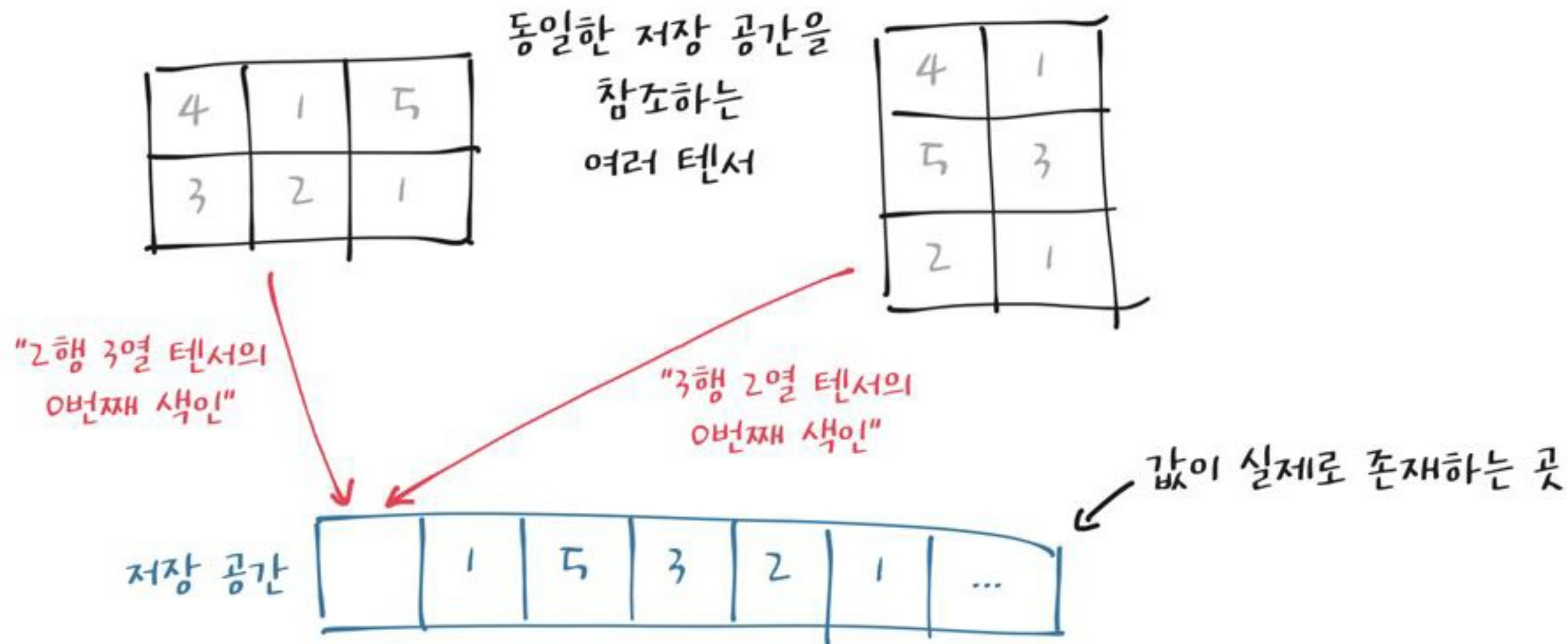


그림 3.4 텐서는 `Storage` 인스턴스에 대한 뷰다.

3.7.1 저장 공간 인덱싱

앞에서 본 2D 포인트로 저장 공간 인덱싱이 어떻게 동작하는지 알아보자. 텐서를 위한 저장 공간은 `.storage` 프로퍼티로 접근할 수 있다.

```
# In[17]:  
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])  
points.storage()
```

```
# Out[17]:  
4.0  
1.0  
5.0  
3.0  
2.0  
1.0  
[torch.FloatTensor of size 6]
```

이 텐서는 세 개의 행과 두 개의 열로 이뤄져 있지만 실제로는 크기가 6인 배열 공간일 뿐이다. 텐서는 주어진 차원 쌍이 실제로 어느 공간에 해당하는지를 알 뿐이다.

따라서 텐서를 거치지 않고 저장 공간을 다음과 같이 직접 접근할 수도 있다.

```
# In[18]:  
points_storage = points.storage()  
points_storage[0]
```

```
# Out[18]:  
4.0
```

```
# In[19]:  
points.storage()[1]
```

```
# Out[19]:  
1.0
```

2차원 텐서라고 해서 저장 공간을 두 개의 인덱스로 2차원처럼 접근하지는 않는다. 차원에 무관하게 실제 저장 공간 레이아웃은 1차원이다.

여기까지 잘 따라왔다면, 저장 공간에 접근해서 값을 바꾸면 참조하고 있는 텐서에서의 내용도 바뀌는 것을 짐작할 수 있을 것이다.

```
# In[20]:
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])
points_storage = points.storage()
points_storage[0] = 2.0
points

# Out[20]: tensor([[2., 1.],
                  [5., 3.],
                  [2., 1.]])
```

3.7.2 저장된 값을 수정하기: 텐서 내부 연산

앞 절에서 소개한 텐서 연산에 이어 Tensor 객체에 대해서만 동작하는 몇 가지 연산을 더 알아보자. 이 연산은 `zero_`처럼 `밑줄(_)`로 끝나는데 연산의 결과로 새 텐서가 넘어오는 대신 기존 텐서의 내용이 바뀐다. 예를 들어 `zero_` 메소드는 입력된 텐서의 모든 요소를 0으로 바꾼다. 밑줄로 끝나지 않는 모든 메소드들은 원래 텐서는 그대로 두고 새로운 텐서를 만들어 넘겨준다.

```
# In[73]:
```

```
a = torch.ones(3, 2)
```

```
# In[74]:
```

```
a.zero_()
```

```
a
```

```
# Out[74]:
```

```
tensor([[0., 0.],  
        [0., 0.],  
        [0., 0.]])
```


3.8 텐서 메타데이터: 사이즈, 오프셋, 스트라이드

저장 공간을 인덱스로 접근하기 위해 텐서는 저장 공간에 포함된 몇 가지 명확한 정보, 즉 사이즈size, 오프셋offset, 스트라이드stride에 의존한다. 그림 3.5는 텐서가 이를 어떻게 사용하는지 보여준다. 텐서의 사이즈(넘파이 용어로는 셰이프shape)는 텐서의 각 차원 별로 들어가는 요소의 수를 표시한 튜플이다.⁷ 저장 공간에 대한 오프셋은 텐서의 첫 번째 요소를 가리키는 색인 값과 동일하다. 스트라이드는 각 차원에서 다음 요소를 가리키고 싶을 때 실제 저장 공간상에서 몇 개의 요소를 건너뛰어야 하는지를 알려주는 숫자다.

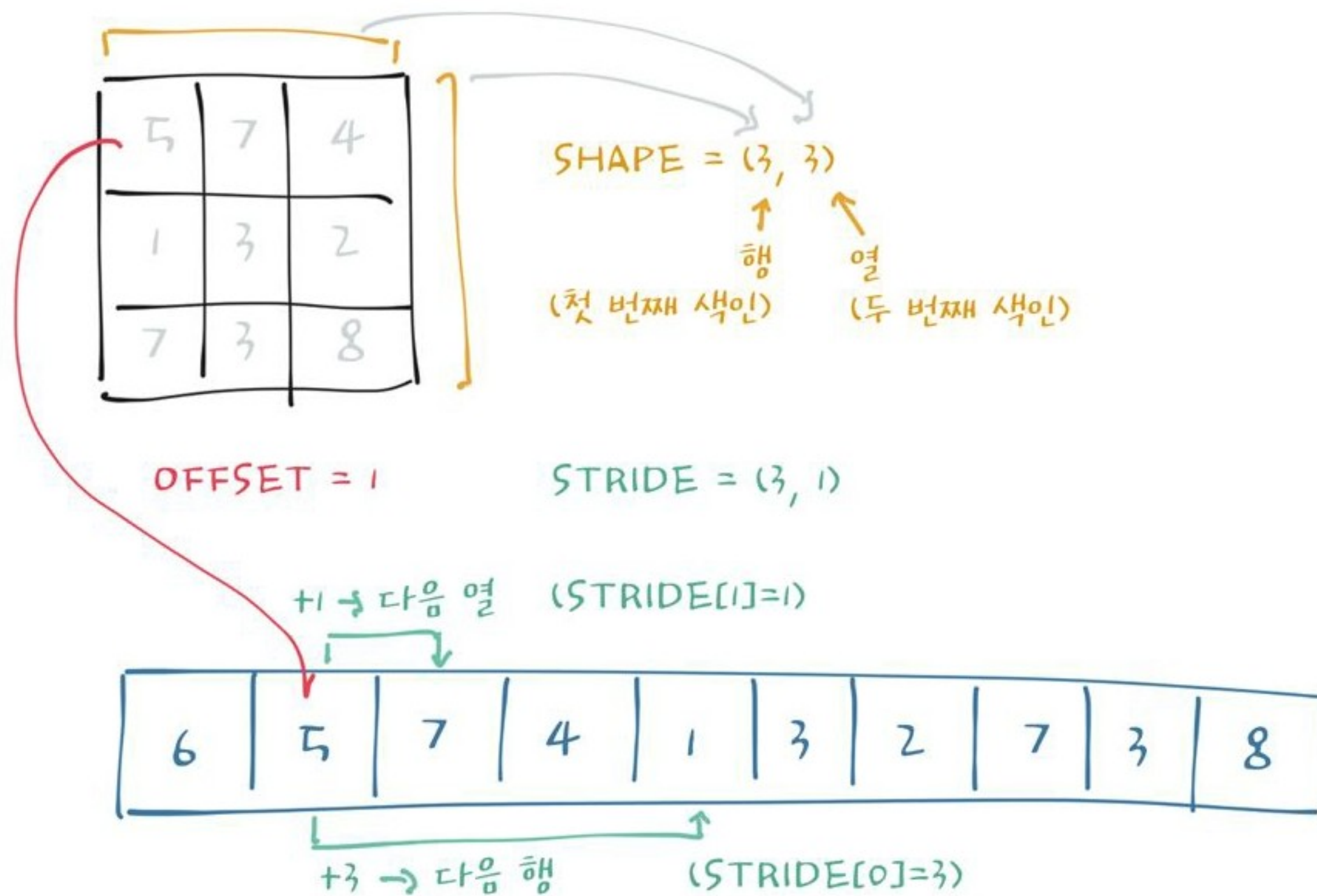


그림 3.5 텐서 오프셋, 사이즈, 스트라이드와의 관계. 그림의 예에서 저장 공간은 텐서로 볼 수 있는 영역보다 더 크다.

3.8.1 다른 텐서의 저장 공간에 대한 뷰 만들기

다음과 같이 대응하는 인덱스를 제공해서 텐서에 들어 있는 두 번째 포인트를 얻을 수 있다.

```
# In[21]:  
points = torch.tensor([4.0, 1.0], [5.0, 3.0], [2.0, 1.0])  
second_point = points[1]  
second_point.storage_offset()
```

```
# Out[21]:  
2
```

```
# In[22]:  
second_point.size()
```

```
# Out[22]:  
torch.Size([2])
```

두 번째 포인트는 저장 공간에서 첫 포인트의 값 두 개 다음이니까 오프셋 2에 해당하는 위치에 있다. 그리고 두 번째 포인트는 1차원이므로 Size 클래스는 하나의 요소를 가지는 객체이고 값으로는 차원 크기가 들어 있다. 이 값이 텐서 객체의 shape 속성값과 동일하니 기억하기 바란다.

```
# In[23]:  
second_point.shape
```

```
# Out[23]:  
torch.Size([2])
```

스트라이드는 값을 가진 튜플인데, 각 차원에서 인덱스를 하나 증가했을 때 저장 공간상에서 몇 개 요소를 건너뛰어야 하는지를 값으로 가진다. Points 텐서는 (2, 1) 스트라이드를 가진다.

```
# In[24]:  
points.stride()
```

```
# Out[24]:  
(2, 1)
```


2차원 텐서에서 요소 i, j 에 접근한다면 저장 공간상으로는 $\text{storage_offset} + \text{stride}[0] * i + \text{stride}[1] * j$ 번째 요소다. 통상 오프셋 값은 0이지만 저장 공간 일부에 접근하는 텐서뷰라면 오프셋은 양수로 지정될 수 있다.

Tensor를 통해 Storage를 간접 접근하는 방식으로 텐서의 전치^{Transpose}나 일부만으로 더 작은 텐서를 만드는 데 필요한 연산을 저렴하게 수행할 수 있다. 왜냐하면 메모리 재할당을 하지 않기 때문이다. 메모리 재할당 대신, 사이즈, 저장소 오프셋, 스트라이드를 변경한 새로운 Tensor 객체를 할당하는 과정을 포함한다.

예제에서 인덱스로 특정 부분만 정의해서 서브텐서^{subtensor}를 만들어봤고 스토리지 오프셋이 증가한 것도 확인했다. 사이즈나 스트라이드 값도 살펴보자.

```
# In[25]:
second_point = points[1]
second_point.size()

# Out[25]:
torch.Size([2])

# In[26]:
second_point.storage_offset()

# Out[26]:
2

# In[27]:
second_point.stride()

# Out[27]:
(1,)
```

여기에서 중요한 점은 예상대로 새 텐서가 원래의 points 텐서보다 하나 작은 차원을 가지지만 여전히 동일한 저장 공간을 가리키고 있다는 점이다. 때문에 새 텐서를 변경하면 원래의 텐서도 바뀐다.

```
# In[28]:
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])
second_point = points[1]
second_point[0] = 10.0
points
```

```
# Out[28]:
tensor([[ 4.,  1.],
        [10.,  3.],
        [ 2.,  1.]])
```

이런 동작 방식이 항상 바람직하지는 않으므로, 결과적으로 서브텐서를 새 텐서로 복제할 수 있다.

```
# In[29]:
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])
second_point = points[1].clone()
second_point[0] = 10.0
points
```

```
# Out[29]:
tensor([[4., 1.],
        [5., 3.],
        [2., 1.]])
```


3.8.2 복사 없이 텐서 전치하기

이제 전치를 해보자. `points` 텐서에는 행마다 개별 포인트가 들어 있고 열에는 X 와 Y 좌표를 가진다. 이를 전치하여 열에 포인트가 들어가도록 만들자. 2차원 텐서들에 대해 `transpose`를 사용해도 되고 다음과 같이 `t` 함수를 사용해도 된다.

```
# In[30]:
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])
points
```

```
# Out[30]:
tensor([[4., 1.],
        [5., 3.],
        [2., 1.]])
```

```
# In[31]:
points_t = points.t()
points_t
```

```
# Out[31]:
tensor([[4., 5., 2.],
        [1., 3., 1.]])
```

두 텐서가 같은 공간을 가리키고 있는지는 다음과 같이 쉽게 확인 가능하다.

```
# In[32]:  
id(points.storage()) == id(points_t.storage())
```

```
# Out[32]:  
True
```

두 텐서는 차원 정보와 스트라이드만 다르므로, 다음과 같은 결과가 나온다.

```
# In[33]:  
points.stride()
```

```
# Out[33]:  
(2, 1)
```

```
# In[34]:  
points_t.stride()
```

```
# Out[34]:  
(1, 2)
```

points에서 인덱스를 하나 올리는 것은 어느 차원인지에 따라 의미가 다르다. points[0,0]에서 points[1,0]으로의 이동은 저장 공간에서 요소 두 개를 건너뛰는 것과 같고 points[0,0]에서 points[0,1]로의 이동은 하나만 건너뛰는 것과 같다. 다른 말로 하면 텐서에서 각 요소는 행 단위로 나뉘어 있다.

points를 points_t로 전치하는 모습을 그림 3.6에서 볼 수 있다. 스트라이드의 요소 순서를 바꾸고 있다. 그다음으로 텐서의 첫 번째 인덱스인 행을 증가시키면 저장 공간상에서 하나만큼 건너뛰고, points에서 열을 따라 이동하던 것과 동일하게 된다. 이것이 전치 연산의 정의다. 새로운 메모리는 할당되지 않으며 전치 연산은 원래 것과 다른 스트라이드 순서를 가진 새로운 Tensor 인스턴스를 만드는 것으로 끝난다.

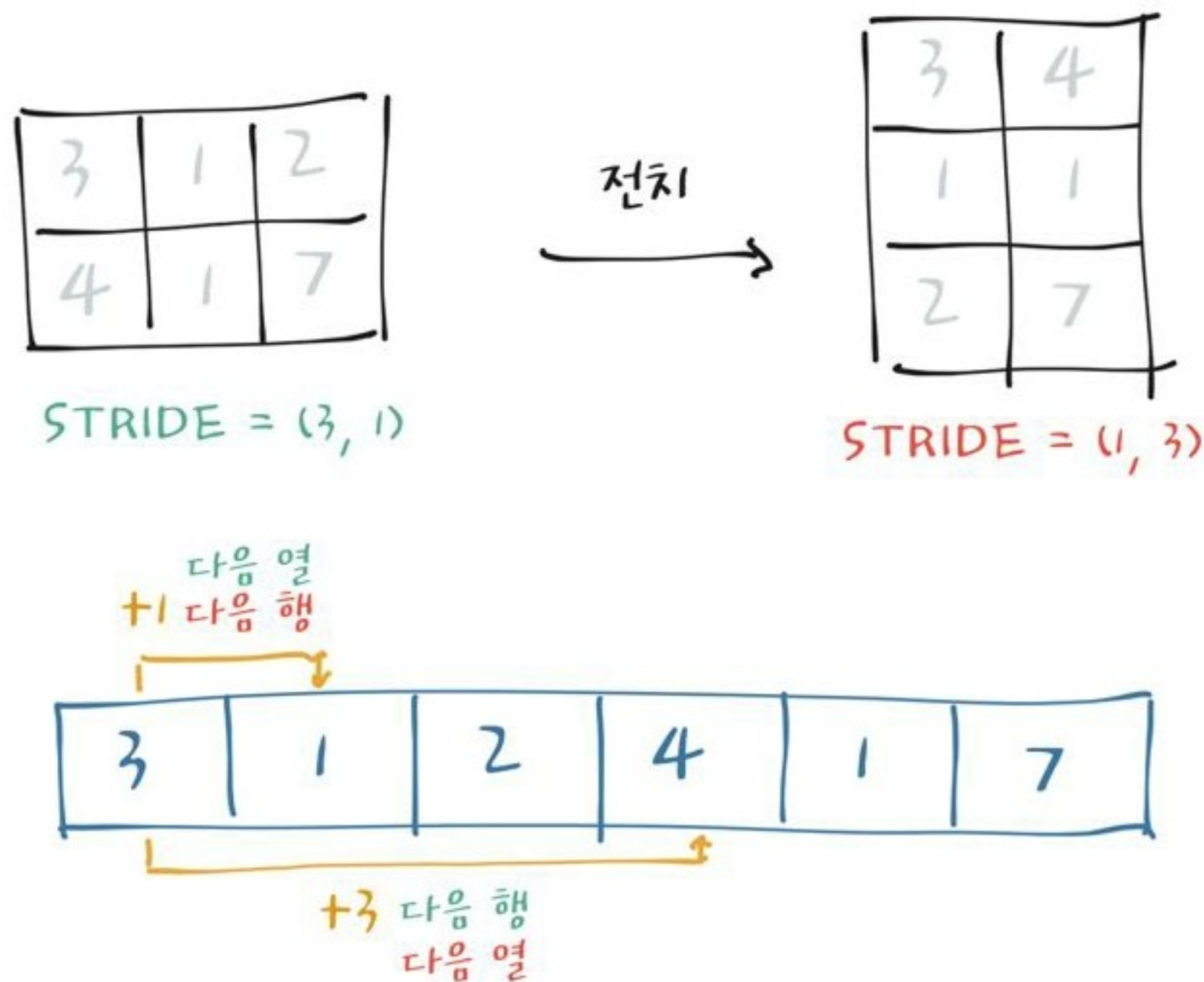


그림 3.6 텐서에 적용된 전치 연산

3.8.3 더 높은 차원에서의 전치 연산

파이토치의 전치 연산은 행렬에만 국한되지 않는다. 다차원 배열에 대해서도 차원 정보와 스트라이드가 바뀔 두 차원을 각각 지정해주면 전치된다.

```
# In[35]:
some_t = torch.ones(3, 4, 5)
transpose_t = some_t.transpose(0, 2)
some_t.shape
```

```
# Out[35]:
torch.Size([3, 4, 5])
```

```
# In[36]:
transpose_t.shape
```

```
# Out[36]:
torch.Size([5, 4, 3])
```

```
# In[37]:
some_t.stride()
```

가장 오른쪽 차원에서 시작해서 증가되는 형태로 저장소에 값이 펼쳐진 텐서(즉 2차원 텐서에서 열을 따라 이동)는 contiguous로 정의된다. 인접한 contiguous 텐서는 값 순회 시 띄엄띄엄 참조하지 않기 때문에 데이터 지역성 data locality 관점에서 CPU 메모리 접근 효율이 좋다. 실제 효율 여부는 이를 사용하는 알고리즘에 따라 다를 수 있다.

```
# Out[37]:
(20, 5, 1)
```

```
# In[38]:
transpose_t.stride()
```

```
# Out[38]:
(1, 5, 20)
```

3.8.4 인접한 텐서

파이토치 텐서 연산 중에는 인접한 텐서에 대해서만 동작하는 경우가 있는데 다음 장에서 살펴볼 view가 그 예다. 이런 경우에 파이토치는 경고성 예외를 던져서 명시적으로 contiguous를 호출하라고 알려준다. 텐서가 이미 인접한 상태라면 contiguous가 실제로 하는 일은 없고 성능에 지장을 주지도 않는다.

다음 예에서 points는 연속된 텐서지만 이를 전치한 결과는 그렇지 않다.

```
# In[39]:
points.is_contiguous()

# Out[39]:
True

# In[40]:
points_t.is_contiguous()

# Out[40]:
False
```

contiguous 메소드를 사용하면 인접하지 않은 텐서를 인접한 텐서로 만들 수도 있다. 텐서 내용은 동일하나, 값의 배치와 스트라이드가 바뀐 텐서가 만들어진다.

```
# In[41]:  
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])  
points_t = points.t()  
points_t
```

```
# Out[41]:  
tensor([[4., 5., 2.],  
        [1., 3., 1.]])
```

```
# In[42]:  
points_t.storage()
```

```
# Out[42]:  
4.0  
1.0  
5.0  
3.0  
2.0  
1.0  
[torch.FloatTensor of size 6]
```

```
# In[43]:  
points_t.stride()
```

```
# Out[43]:  
(1, 2)
```

```
# In[44]:
points_t_cont = points_t.contiguous()
points_t_cont
```

```
# Out[44]:
tensor([[4., 5., 2.],
        [1., 3., 1.]])
```

```
# In[45]:
points_t_cont.stride()
```

```
# Out[45]:
(3, 1)
```

```
# In[46]:
points_t_cont.storage()
```

```
# Out[46]:
4.0
5.0
2.0
1.0
3.0
1.0
[torch.FloatTensor of size 6]
```


항목을 열 단위로 배치하기 위해 새 저장 공간을 재편성한 것에 주목하자. 새로운 배치에 따라 스트라이드도 바뀌었다.

앞서 설명한 부분을 다시 상기하기 위해 그림 3.7을 살펴보자. 이제 텐서가 어떻게 만들어지는지 더 잘 파악할 수 있다.

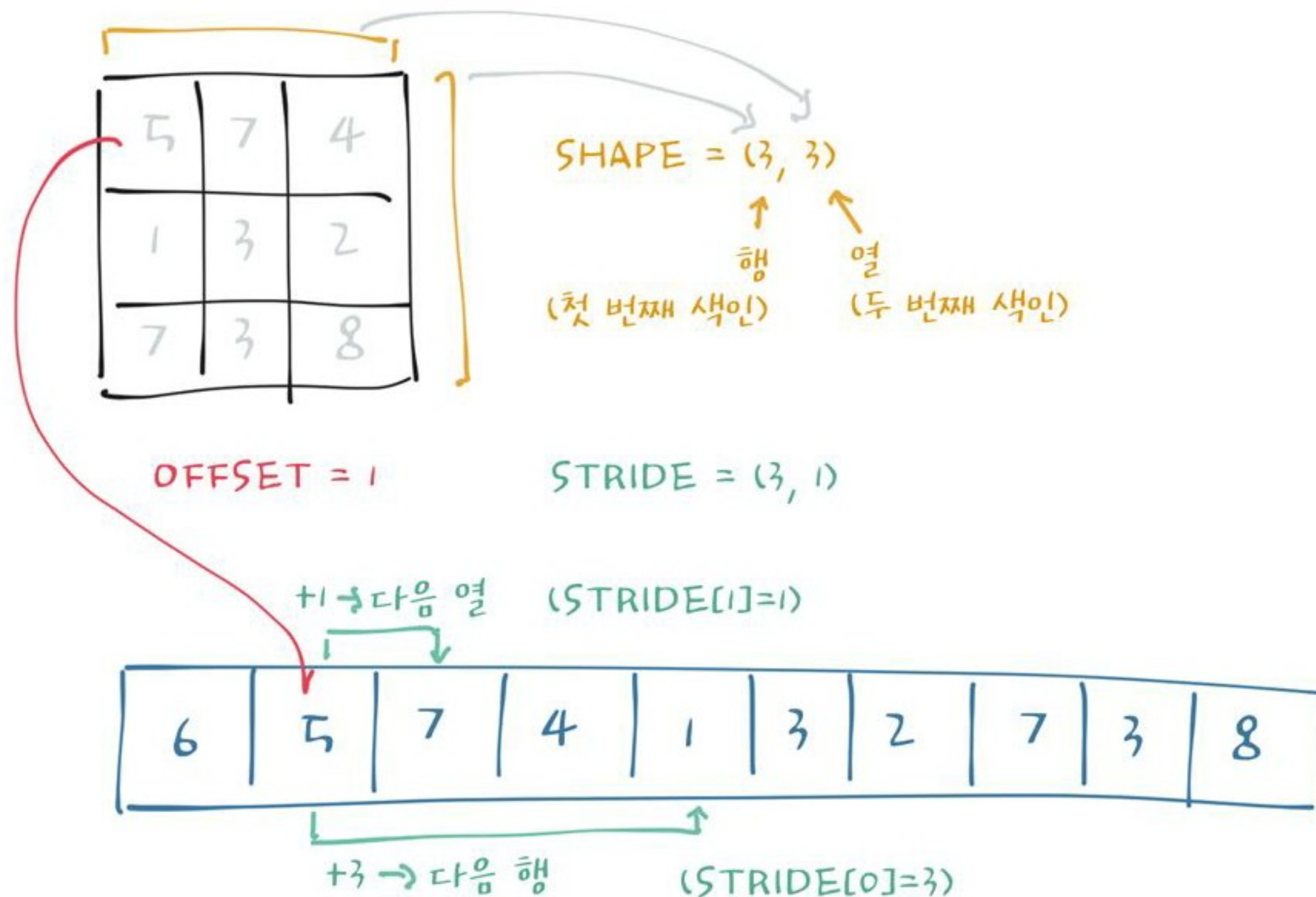


그림 3.7 텐서 오프셋과 사이즈, 스트라이드 간의 관계. 이 텐서는 더 큰 텐서의 저장 공간의 일부를 가리키는 뷰 역할을 하는 텐서다.

3.9 텐서를 GPU로 옮기기

지금까지 살펴본 텐서 저장 공간은 CPU 메모리에 대한 것이었다. 파이토치의 텐서는 CPU 외 다른 프로세서인 GPU를 위한 메모리 공간에도 저장할 수 있다. 모든 파이토치 텐서는 여러 GPU 중에 특정 GPU 메모리 영역으로 이동할 수 있고 이렇게 대량 병렬 처리 연산을 빠르게 할 수 있다. 이때의 텐서 연산은 파이토치가 지원하는 GPU 전용 루틴을 사용한다.

다양한 종류의 GPU를 지원하는 파이토치

2019년 중반까지 배포된 주요 파이토치 버전은 CUDA 지원 GPU만 가능했다. 파이토치는 AMD의 ROCm(<https://rocm.github.io>)에서 돌릴 수 있고 마스터 저장소에서 지원하지만, 파이토치를 AMD에서 사용하려면 직접 컴파일해야 한다. (정상적인 빌드 과정 전에 GPU 코드 변환을 위해 `tools/amd_build/build_amd.py`를 실행해야 한다) 구글의 텐서 프로세싱 유닛인 TPU 지원은 현재 진행 중이며(<https://github.com/pytorch/xla>) 프로토타입이 구글 코랩(<https://colab.research.google.com>)에 공개되어 있다. 그 밖에 OpenCL 같은 여러 GPU 기술의 자료구조나 커널 구현 계획은 현재 없다.

3.9.1 텐서 디바이스 속성 관리

dtype 외에 파이토치의 Tensor는 device라는 인자로 텐서 데이터가 실제 컴퓨터의 어디에 위치할지도 지정할 수 있다. 다음은 생성자에 device 인자를 지정하여 텐서를 GPU에 만드는 방법이다.

```
# In[64]:  
points_gpu = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]], device='cuda')
```

아니면 CPU에 만들어진 텐서를 to 메소드를 활용해 GPU로 옮길 수도 있다.

```
# In[65]:  
points_gpu = points.to(device='cuda')
```

이렇게 해서 숫자값이 동일한 새 텐서를 받을 수 있는데 이 텐서가 위치하는 장소는 일반 시스템의 RAM이 아닌 GPU의 RAM이다. 이제 GPU에 데이터가 저장되었으니 앞에서 설명했던 것처럼 수학 연산이 얼마나 빨라지는지 확인해보자. 대부분의 경우 CPU에 있는 텐서나 GPU에 있는 텐서 모두 동일한 사용자 API를 제공하기 때문에 복잡한 대량의 숫자 연산이 일어나는 정확한 장소에 상관없이 코드 작성이 훨씬 더 쉬워진다.

만일 컴퓨터에 GPU가 둘 이상 탑재되어 있다면 어떤 GPU에서 동작시킬지는 GPU를 0번부터 시작하는 정수로 지명하면 된다.

```
# In[66]:  
points_gpu = points.to(device='cuda:0')
```

이제부터 모든 요소에 특정 상수를 곱하는 등의 텐서 연산은 GPU에서 수행된다.

```
# In[67]:  
points = 2 * points  ← CPU에서 실행하는 곱셈  
points_gpu = 2 * points.to(device='cuda')  ← GPU에서 실행하는 곱셈
```

계산 완료 후 `points_gpu` 텐서는 CPU 영역으로 옮겨지지 않으니 주의하자. 이 코드는 다음 단계를 실행한다.

1. `points` 텐서를 GPU에 복사한다.
2. GPU에 새 텐서를 할당한 후 곱셈 결과를 저장한다.
3. GPU 텐서 핸들을 반환한다.

그러므로 만일 우리가 결과에 상수를 더한다면 다음과 같다.

```
# In[68]:  
points_gpu = points_gpu + 4
```

연산을 여전히 GPU에서 수행하며 출력하거나 결과 텐서 값에 접근하지 않는 이상, CPU에는 아무 정보도 전달되지 않는다. 텐서를 다시 CPU로 옮기려면 `to` 메소드에 `cpu` 인자를 전달한다.

```
# In[69]:  
points_cpu = points_gpu.to(device='cpu')
```

같은 동작을 위해 `to` 메소드 대신 `cpu`나 `cuda`라는 축약 메소드를 사용할 수도 있다.

```
# In[70]:  
points_gpu = points.cuda()  ← 기본으로 0번 GPU에 복사함  
points_gpu = points.cuda(0)  
points_cpu = points_gpu.cpu()
```

`to` 메소드를 사용할 때 `device`와 `dtype` 인자를 동시에 사용해서 데이터 타입과 데이터 위치를 동시에 변경할 수 있다.

3.10 넘파이 호환

앞에서 여러 번 넘파이NumPy를 언급했다. 이 책을 읽기 위해 넘파이를 미리 알아둬야 한다고 얘기하지는 않았지만, 넘파이는 파이썬 데이터과학 생태계에서 매우 독보적이기 때문에 모쪼록 친숙해지기를 강력히 추천한다. 파이토치 텐서와 넘파이 배열의 변환은 매우 효율적으로 이뤄진다. 이런 변환을 통해 파이썬 생태계에서 넘파이 배열 타입을 위해 만들어진 다양한 기능들을 마음껏 누릴 수 있다. 넘파이 배열과의 제로카피zero-copy 수준의 상호 변환은 파이썬 버퍼 프로토콜이라는 저장 시스템 덕분이다(<https://docs.python.org/3/c-api/buffer.html>).

points 텐서를 넘파이 배열로 바꾸는 방법은 다음과 같이 단순하다.

```
# In[55]:
```

```
points = torch.ones(3, 4)
```

```
points_np = points.numpy()
```

```
points_np
```

```
# Out[55]:
```

```
array([[1., 1., 1., 1.],  
       [1., 1., 1., 1.],  
       [1., 1., 1., 1.]], dtype=float32)
```


이렇게 하면 원하는 사이즈와 차원 정보, 타입을 가진 다차원 넘파이 배열을 얻을 수 있다. 흥미롭게도 이 배열은 텐서의 저장 공간을 그대로 버퍼로 사용한다. 즉 데이터가 CPU RAM 영역에 있는 한, numpy 메소드를 아무 추가 비용 없이 실행할 수 있다. 넘파이 배열을 수정하면 원래 텐서에도 그대로 적용된다. 만일 텐서가 GPU에 할당되어 있다면 파이토치는 넘파이 배열로의 변환 과정에서 CPU 영역으로 텐서 내용을 복사해서 배열을 만든다.

이번에는 반대로, 넘파이 배열을 파이토치 텐서로 만들어보자.

```
# In[56]:  
points = torch.from_numpy(points_np)
```

동일하게 버퍼를 공유하는 전략으로 동일한 공간에 위치한 텐서를 얻게 된다.

노트

파이토치의 기본 숫자 타입이 32비트 부동소수점인 반면, 넘파이는 64비트다. 3.5.2절에서 설명한 것처럼 우리는 기본적으로 32비트 부동소수점을 사용하므로, 변환 후에는 텐서가 dtype torch.float인지 확인할 필요가 있다.

3.11 일반화된 텐서도 텐서다

이번 3장뿐 아니라 이 책 전반과 상당수의 애플리케이션에서 볼 수 있듯이 텐서는 다차원 배열이다. 하지만 아래쪽을 파고들다 보면 한 가지 반전이 있다. 실제 내부 구현과 3.6절의 텐서 API는 사실 분리되어 있다. API 명세를 만족하기만 하면 실제 구현 상세는 몰라도 텐서로 간주할 수 있다는 얘기다!

파이토치는 우리의 텐서가 CPU에 있거나 GPU에 있거나 상관없이 알맞는 연산 함수를 호출한다. 이는 **분배**^{dispatching} 메커니즘에 의한 것인데 이 메커니즘은 사용자 API 입장에서 해당 텐서 타입에 맞는 백엔드 함수를 골라서 실행한다. 이쯤 되면 당연히 다른 형태의 텐서 구현이 존재한다는 것도 짐작할 수 있다. 구글의 TPU 같은 하드웨어 장치에 특화된 텐서나 우리가 지금까지 봐온 밀집 배열과는 다른 데이터 표현 전략을 가진 텐서가 그 예다. 희소^{sparse} 텐서는 0이 아닌 경우만 인덱스와 값을 만들어 저장한다. 그림 3.8의 왼쪽 그림에서 보이는 파이토치 분배기^{dispatcher}는 확장 가능하도록 설계되었으며 내부의 스위치 로직으로 그림 3.8의 오른쪽 그림처럼 백엔드 별로 구현된 각각의 코드 중에 선택하여 다양한 숫자 타입을 다룰 수 있게 되어 있다.

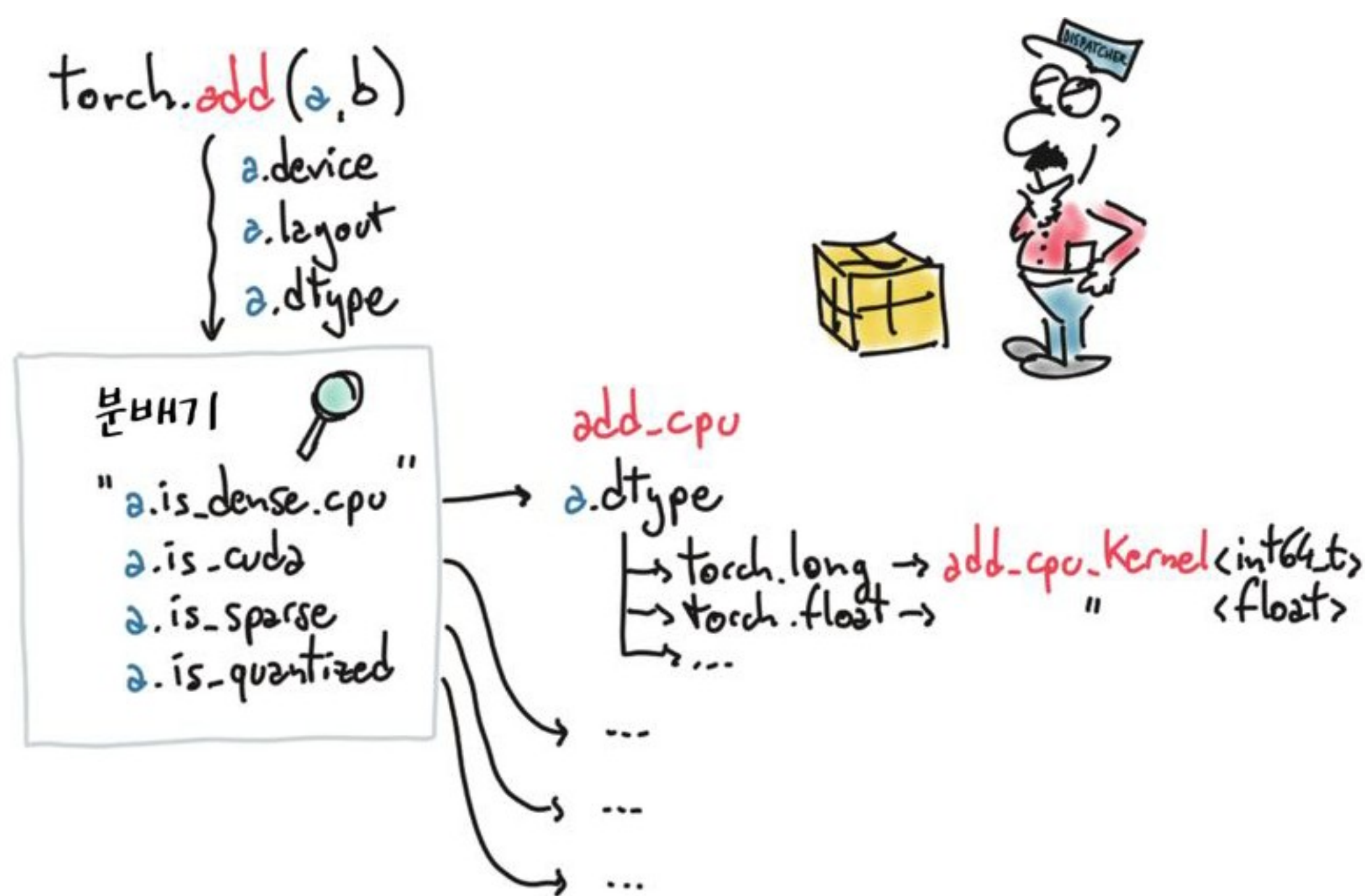


그림 3.8 파이토치 분배기는 핵심 내부 구조 중 하나다.

15장에서는 양자화^{quantized}된 텐서도 볼 수 있다. 이는 백엔드 연산을 특별하게 수행하도록 구현된 텐서다. 또 어떤 경우에는 우리가 사용하는 텐서를 dense 혹은 strided라는 다양한 메모리 배치 방식으로 구별하기도 한다.

파이토치가 다양한 하드웨어와 애플리케이션을 지원하며 텐서의 종류도 다양해졌다. 앞으로도 사람들은 파이토치로 자료 표현과 연산 수행에 다양한 시도를 하게 될 것이고 따라서 새로운 종류는 계속 늘어날 것으로 보인다.

3.12 텐서 직렬화

텐서를 직접 만들어보겠다는 생각은 꽤 훌륭하다. 하지만 그 안에 담긴 데이터가 중요하다면 이를 파일에 보관했다가 나중에 다시 읽어들이고 싶을 것이다. 누구도 프로그램을 실행할 때마다 모델을 처음부터 재훈련시키기를 원하지는 않을 것이다! 파이토치는 텐서 객체를 직렬화하기 위해 내부적으로 pickle을 사용하며, 저장 공간을 위한 전용 직렬화 코드를 가지고 있다. 다음은 우리의 points 텐서를 ourpoints.t 파일에 저장하는 코드다.


```
# In[57]:  
torch.save(points, '../data/plch3/ourpoints.t')
```

혹은 파일 이름 대신 파일 디스크립터(descriptor)를 넣어도 된다.

```
# In[58]:  
with open('../data/plch3/ourpoints.t','wb') as f:  
    torch.save(points, f)
```

텐서로 불러들일 때는 한 줄이면 된다.

```
# In[59]:  
points = torch.load('../data/plch3/ourpoints.t')
```

아니면 다음처럼 해도 된다.

```
# In[60]:  
with open('../data/plch3/ourpoints.t','rb') as f:  
    points = torch.load(f)
```

파이토치로만 읽을 예정이면 이처럼 간단하게 텐서를 저장할 수 있다. 다만 파일 포맷 자체가 호환되지 않기 때문에 파이토치가 아닌 다른 소프트웨어로는 읽을 수 없다. 경우에 따라서는 제약이 될 수도 있는데 이런 요건이 걸림이 된다면 텐서를 호환 가능하도록 저장하는 법을 배워야 한다. 이어서 저장 방법을 살펴보자.

3.12.1 h5py로 HDF5 병렬화하기

경우마다 다르지만 이미 다른 라이브러리에 의존하는 시스템이 있는 상태에서 파이토치를 도입하는 경우라면 텐서를 호환 가능한 형태로 저장해야 할 가능성이 높을 것이다. 새 프로젝트에서는 이럴 필요가 거의 없다.

호환성이 중요하다면, 먼저 HDF5 포맷을 위한 라이브러리가 필요하다(www.hdfgroup.org/solutions/hdf5). HDF5는 이식성이 높고, 광범위하게 지원되는, 중첩된 키-값 디렉터리에서 직렬화된 정형 다차원 배열을 표현하는 포맷이다. 파이썬은 h5py 라이브러리를 통해 HDF5 포맷을 지원한다(www.h5py.org). 이 라이브러리는 넘파이 배열 형태로 전달하거나 반환받을 수 있다. 설치하는 다음과 같이 할 수 있다.

```
$ conda install h5py
```

이제 points 텐서를 (앞에서 설명한 바와 같이 아무런 추가 연산 비용 없이) 넘파이 배열로 변환해서 create_dataset 함수에 전달하면 저장할 수 있다.

```
# In[61]:  
import h5py  
  
f = h5py.File('../data/p1ch3/ourpoints.hdf5', 'w')  
dset = f.create_dataset('coords', data=points.numpy())  
f.close()
```


여기에서 'coords'가 HDF5 파일의 키에 해당하는데 키와 키를 중첩^{nested}시킬 수도 있다. HDF5 포맷의 재미있는 특징 중 하나는 데이터가 디스크에 있는 상태에서 원하는 요소만 인덱스로 직접 접근하도록 지원해주는 점이다. 예를 들어 데이터셋의 가장 뒤 포인트 두 개만 읽고 싶다고 가정해보자.

```
# In[62]:  
f = h5py.File('../data/p1ch3/ourpoints.hdf5', 'r')  
dset = f['coords']  
last_points = dset[-2:]
```

파일이 열리거나 데이터셋이 필요할 때 아직 실제 데이터는 읽지 않은 상태다. 맨끝이나 뒤에서 두 번째 있는 요소를 읽기 전까지는 아직 디스크에 존재한다. h5py는 위 코드에서 맨끝 요소 두 개에 접근해서 넘파이 배열 API와 동일하게 동작하는 유사 넘파이 배열 객체를 넘겨준다.

이것을 `torch.from_numpy` 함수에 그대로 넘겨주면 곧바로 텐서를 얻을 수 있다. 이때에는 데이터가 텐서의 저장 공간으로 복사된 상태다.

```
# In[63]:  
last_points = torch.from_numpy(dset[-2:])  
f.close()
```

데이터를 읽었다면 파일을 닫아야 한다. HDF5 파일을 닫으면 데이터셋이 무효화되므로 이후부터는 `dset` 접근시 예외가 발생한다. 설명한 순서대로 실행했다면 이제 `last_points` 텐서를 사용할 수 있다.

3.13 결론

부동소수점 수 표현을 위해 필요한 모든 것을 살펴보았다. 텐서의 뷰 만들기, 다른 텐서로 텐서 인덱싱하기, 다른 사이즈나 차원 정보를 가지는 텐서 사이에서 요소 단위 연산을 단순화시켜주는 브로드캐스팅하기 등에 대한 자세한 내용은 필요한 경우에 더 살펴보겠다.

다음 4장에서는 파이토치로 실제 데이터를 표현하는 법을 배울 것이다. 간단한 테이블 데이터를 가지고 조금씩 다듬어 나갈 예정이다. 이 과정을 통해 텐서에 대해 좀더 자세히 알게 될 것이다.

3.15 핵심 요약

- 신경망은 부동소수점 표현을 다른 부동소수점 표현으로 바꿔준다. 입력이나 출력 부분의 값들은 사람이 확인 가능하지만 중간 단계의 값은 파악하기 어렵다.
- 부동소수점 표현은 텐서에 저장한다.
- 텐서는 다차원 배열이며 파이토치의 기본 자료구조다.
- 파이토치는 텐서의 생성이나 조작 그리고 수학 연산 같은 다양한 표준 라이브러리를 제공한다.
- 텐서는 병렬화해 디스크에 저장하거나 반대로 불러올 수 있다.
- 파이토치의 모든 텐서 연산은 코드 변경 없이 쉽게 CPU에서 혹은 GPU에서 실행할 수 있다.
- 파이토치에서 바꿔치기 연산을 수행하는 함수 이름은 `Tensor.sqrt_`처럼 밑줄(_)로 끝난다.